外卖应用后端架构方案

架构提案

为 星河餐饮有限公司 准备

日期	修订	产品	目录核对时间
2026年10月31日	3	14	2026-10-31

使用 Architecture Builder 生成

由 AI 生成。构建前请与解决方案架构师复核。

执行摘要

星河餐饮有限公司正在构建全新的外卖应用后端，以支撑骑手与顾客的移动端体验。当前团队面临的核心挑战在于：需要在不依赖 Kubernetes 的前提下运行十二个微服务，同时为明年预计五倍的流量增长做好准备。本方案专为这一业务目标设计，确保技术投入直接转化为订单处理能力和用户体验的提升。

我们采用全托管的 Serverless 架构，让团队无需学习复杂的容器编排即可实现微服务的自动扩缩容。从用户点击菜单到骑手接单，每一个环节都针对高并发场景进行了优化，包括引入专用缓存加速菜单加载、通过消息队列解耦订单事件以避免高峰期系统阻塞。

对管理层而言，这意味着更低的运维人力成本、更快的业务上线速度，以及在流量洪峰期间依然稳定的服务质量。架构中的每一层都具备弹性能力，您只需为实际消耗的资源付费，避免为未来的不确定性提前买单。

本方案实现的价值

零门槛的微服务运营	团队无需掌握 Kubernetes 即可管理十二个微服务。Serverless 模式将基础设施的维护责任转移给云平台，开发人员可以专注于外卖业务逻辑的迭代，缩短新功能的上市时间。
丝滑的顾客点餐体验	通过边缘加速、API 网关路由优化以及专属的菜单缓存层，顾客浏览菜品和下单的响应时间将大幅降低。在竞争激烈的外卖市场，更快的加载速度意味着更高的转化率和复购率。
从容应对五倍增长的弹性账单	架构的计算、缓存、数据库和消息队列均具备自动伸缩能力。您无需为了明年的预期峰值而在今天就购买大量闲置资源，真正实现按业务量付费，将 IT 成本与营收增长保持健康的线性关系。

您提出的需求，以及设计如何应对

列出沟通中确认的各项优先级、来源，以及架构中满足这些需求的部分。

在不使用 Kubernetes 的情况下运行和管理十二个微服务

“团队没有 Kubernetes 经验，请不要使用 Kubernetes。”

方案如何满足该需求

我们选择 Serverless App Engine (SAE) 作为计算底座。它屏蔽了底层集群管理的复杂性，您的团队可以直接部署代码或镜像，平台会自动处理实例的生命周期。这消除了招聘 Kubernetes 专家的成本，也避免了因操作失误导致的服务中断风险。

涉及产品

Serverless App Engine (SAE)

为餐厅菜单增加缓存以提升读取速度

“为餐厅菜单加一个缓存。”

方案如何满足该需求

菜单是外卖应用中访问频率最高的数据。我们在微服务与数据库之间引入了 Tair 缓存节点，将热门菜品信息驻留在内存中。这能显著降低数据库压力，让顾客在滑动菜单时获得毫秒级的响应体验，直接减少因等待而流失的订单。

涉及产品

Tair (Redis® OSS-Compatible)

架构需支持明年五倍的流量弹性扩展

“预计明年流量会增长 5 倍，请让架构能够扩展。”

方案如何满足该需求

应对五倍增长不能依靠临时堆砌服务器。我们为 SAE、Tair、PolarDB 和 RocketMQ 全部配置了基于指标的自动扩展策略。当流量上升时，系统会自动增加处理能力；流量回落后自动释放资源。这确保了您在业务爆发期不会丢失订单，在平稳期不会浪费预算。

涉及产品

Serverless App Engine (SAE)、Tair (Redis® OSS-Compatible)、PolarDB、ApsaraMQ for RocketMQ

稳定处理订单事件，防止高峰期系统雪崩

根据您的回答推断

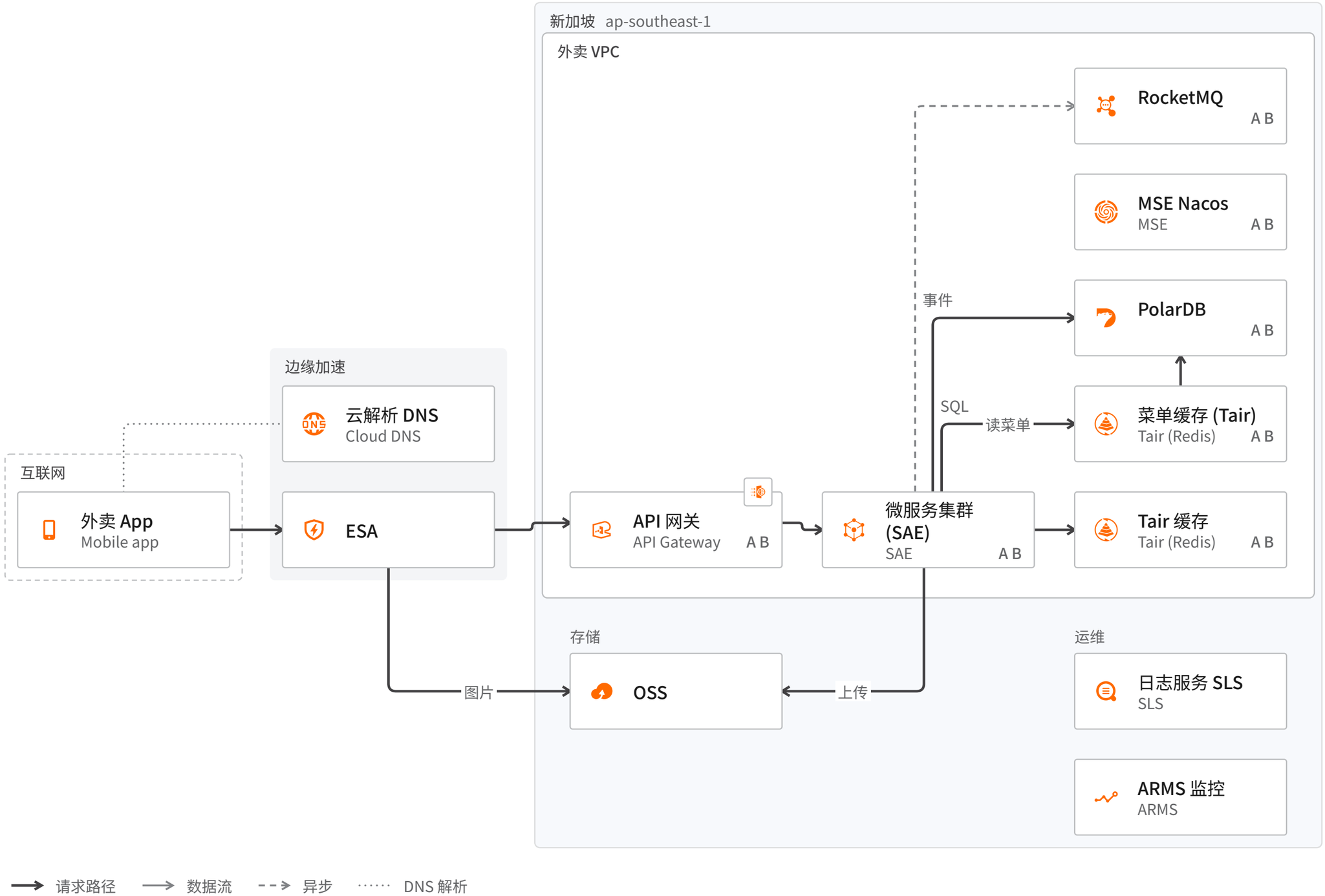
方案如何满足该需求

外卖业务的订单具有明显的波峰波谷特征（如午晚餐高峰）。如果所有订单请求直接冲击数据库，极易导致系统崩溃。我们通过 ApsaraMQ for RocketMQ 将订单的接收与后续处理异步解耦，即使瞬间涌入大量订单，系统也能像蓄水池一样平滑消化，保障交易链路的可靠。

涉及产品

ApsaraMQ for RocketMQ

架构图


新加坡 ap-southeast-1
外卖 VPC
RocketMQ
A B
MSE Nacos
MSE
A B
事件
PolarDB
A B
SQL
读菜单
菜单缓存 (Tair)
Tair (Redis)
A B
边缘加速
云解析 DNS
Cloud DNS
互联网
外卖 App
Mobile app
ESA
API 网关
API Gateway
A B
微服务集群 (SAE)
SAE
A B
Tair 缓存
Tair (Redis)
A B
存储
图片
OSS
上传
运维
日志服务 SLS
SLS
ARMS 监控
ARMS
请求路径
数据流
异步
DNS 解析

组件

列出架构图中的每个产品、其入选原因以及官方文档链接。

Mobile app

外卖 App
互联网

选用原因
这是骑手和顾客与您业务交互的唯一触点。整个后端架构的设计初衷就是为了让这个移动端应用在任何网络环境下都能快速、稳定地获取数据并完成下单。

功能说明
Native iOS or Android app calling the backend over the internet.

官方文档
暂无文档链接

Alibaba Cloud DNS

云解析 DNS
边缘加速 · Domain Names and Websites

选用原因
Alibaba Cloud DNS 负责将用户的 API 请求精准导向离他们最近的边缘节点。如果没有它，用户可能会连接到延迟较高的服务器，直接影响打开应用的速度。

功能说明
Authoritative public DNS for your domains; a resolve hop, not on the data path.

官方文档
www.alibabacloud.com/product/dns

Edge Security Acceleration (ESA)

边缘加速 · Networking & CDN

选用原因
Edge Security Acceleration (ESA) 在靠近用户的地方加速 API 请求并抵御 DDoS 攻击。对于外卖应用而言，恶意流量不仅消耗带宽成本，还可能导致正常用户在饭点无法下单，ESA 保护了您的营收通道。

功能说明
Global edge: caching, dynamic acceleration, WAF, DDoS defense, bot management and edge functions.

官方文档
www.alibabacloud.com/product/esa

Cloud-native API Gateway

API 网关
新加坡 (ap-southeast-1) · 外卖 VPC · 可用区 A, B

选用原因
Cloud-native API Gateway 是移动应用到十二个微服务的统一调度中心。它将复杂的内部服务地址对外隐藏，不仅简化了移动端的开发对接，还能集中控制限流和鉴权，防止某个接口被滥用拖垮全局。

功能说明
Managed Envoy/Higress gateway for REST, HTTP and WebSocket APIs with auth and throttling.

官方文档
www.alibabacloud.com/help/en/api-gateway/cloud-native-api-gateway/

Web Application Firewall (WAF)

新加坡 (ap-southeast-1) · 外卖 VPC · 附加于 API 网关

选用原因

Web Application Firewall (WAF) 专门拦截注入攻击和恶意爬虫。外卖平台的菜品数据和价格信息是核心资产，WAF 能防止恶意爬虫批量抓取您的菜单，同时阻断可能导致数据泄露的安全漏洞。

功能说明

WAF 3.0: blocks OWASP attacks, CC floods and bots via cloud-native or CNAME access.

官方文档

www.alibabacloud.com/product/web-application-firewall

Serverless App Engine (SAE)

微服务集群 (SAE)

新加坡 (ap-southeast-1) · 外卖 VPC · 可用区 A, B

选用原因

Serverless App Engine (SAE) 是本方案的核心计算引擎。它直接满足了团队不使用 Kubernetes 的要求，同时内置的自动扩缩容能力（根据 CPU 指标从 12 个实例扩展到 60 个）是应对明年五倍流量增长的关键免运维方案。若不使用 SAE，您将不得不组建专门的运维团队来手动管理服务器。

功能说明

Serverless PaaS for containers and Java apps, without servers or Kubernetes to run.

官方文档

www.alibabacloud.com/product/severless-application-engine

ApsaraMQ for RocketMQ

新加坡 (ap-southeast-1) · 外卖 VPC · 可用区 A, B

选用原因

ApsaraMQ for RocketMQ 是订单事件的缓冲阀。在五倍流量冲击下，它能确保每一笔订单都不丢失，并让后端服务按照自己的节奏处理。省略它将导致高峰期数据库过载，直接造成丢单。

功能说明

Managed RocketMQ 5.x for transactional, scheduled and ordered messaging.

官方文档

www.alibabacloud.com/product/mq

Microservices Engine (MSE)

MSE Nacos

新加坡 (ap-southeast-1) · 外卖 VPC · 可用区 A, B

选用原因

Microservices Engine (MSE) 提供了 Nacos 服务注册与配置管理能力。当十二个微服务在 SAE 上动态扩缩容时，MSE 确保它们能互相找到对方并获取最新配置。没有它，微服务之间将陷入混乱的调用失败。

功能说明

Managed Nacos or ZooKeeper registry and config, plus microservice governance.

官方文档

www.alibabacloud.com/product/microservices-engine

PolarDB

新加坡 (ap-southeast-1) · 外卖 VPC · 可用区 A, B

选用原因

PolarDB 是最终的数据大本营。其计算存储分离的架构允许在秒级增加只读节点，完美匹配五倍读流量的增长需求。传统数据库在扩容时需要停机迁移，这对于不能中断的外卖业务是不可接受的。

功能说明

Cloud-native relational database (MySQL, PostgreSQL, Oracle-compatible) with compute-storage separation.

官方文档

www.alibabacloud.com/product/polardb

Tair (Redis® OSS-Compatible)

Database

选用原因

Tair (Redis® OSS-Compatible) 承载了您明确要求的菜单缓存功能，并通过升级分片和增加只读副本来支撑五倍的并发读写。如果不加这一层，所有的菜单查询都会压到数据库上，导致点餐页面卡顿。

功能说明

Managed Redis-compatible in-memory database and cache, plus Tair enterprise features.

官方文档

www.alibabacloud.com/product/tair

Object Storage Service (OSS)

新加坡 (ap-southeast-1) · 区域服务 · Storage

选用原因

Object Storage Service (OSS) 用于存放海量的菜品图片和静态资源。将这些大文件从应用服务器剥离出来，不仅降低了微服务的负载，还利用对象存储的低成本特性节省了整体 IT 开支。

功能说明

Durable object storage for files, media, backups and data lakes; S3-compatible.

官方文档

www.alibabacloud.com/product/object-storage-service

Simple Log Service (SLS)

日志服务 SLS

新加坡 (ap-southeast-1) · 区域服务 · Storage

选用原因

Simple Log Service (SLS) 集中收集十二个微服务的日志。当一笔订单出现异常时，开发团队需要跨多个服务排查问题。没有 SLS，排查一次线上故障可能需要数小时，严重影响业务恢复速度。

功能说明

Collects, searches, analyzes with SQL and alerts on logs, metrics and traces.

官方文档

www.alibabacloud.com/product/log-service

Application Real-Time Monitoring Service (ARMS)

ARMS 监控

新加坡 (ap-southeast-1) · 区域服务 · Middleware

选用原因

Application Real-Time Monitoring Service (ARMS) 提供微服务链路追踪和性能监控。它能直观告诉您哪个接口变慢了、哪个服务报错了。在流量翻五倍的背景下，缺乏 ARMS 意味着您对系统的健康状况处于盲区。

功能说明

APM, browser and synthetic monitoring, tracing and alerting for applications.

官方文档

www.alibabacloud.com/product/arms

考量因素与后续步骤

权衡取舍及应对措施

完全放弃 Kubernetes 意味着牺牲了部分底层调度的灵活性和跨云移植的便利性，团队将深度绑定阿里云的 Serverless 生态。

应对措施

通过采用标准的微服务架构（REST API 和通用消息队列），业务代码本身保持了无状态和平台无关性。未来若业务规模达到必须自建集群的程度，代码可平滑迁移，当前的 SAE 方案是兼顾团队现状与业务增速的最佳平衡点。

为了应对五倍流量，我们在缓存、数据库和消息队列上都启用了高级别的弹性规格和集群版配置，这在流量低谷期会产生比基础架构略高的固定底座成本。

应对措施

SAE 和 PolarDB 的自动伸缩机制会在低峰期自动缩减计算节点，将闲置成本降至最低。同时，Tair 的缓存命中率提升大幅减少了数据库的查询开销，整体上是用少量的冗余成本换取了高峰期不宕机的业务保障。

后续步骤

1. 安排一次面向开发团队的技术对齐会议，演示 SAE 的部署流程和 MSE 的配置方式，验证无 Kubernetes 环境下的开发体验。
2. 基于历史订单数据进行一次压力测试模拟，确认 SAE 从 12 扩至 60 个实例以及 PolarDB 增加只读节点的触发阈值是否符合五倍增长的业务预期。
3. 梳理现有的菜单查询接口，制定 Tair 缓存的接入计划与数据过期策略，确保缓存上线后顾客端能立即感受到加载速度的提升。

设计记录

方案的基础依据：包括沟通中固定的需求、约束条件、假设、待决问题，以及首次绘制时的备注。

设计详情

我会用 SAE（无需 Kubernetes）来运行 12 个微服务，通过 API 网关暴露 REST API，并用 RocketMQ 处理订单事件。 12 个微服务部署在 SAE 上，通过 MSE Nacos 做服务注册与配置管理。 订单事件通过 RocketMQ 异步解耦，缓存用 Tair，数据库用 PolarDB。

14 个产品，14 个连接，3 次修订

已固定的需求

团队无 K8s 经验，不使用 Kubernetes	满足 在修订 1 中已固定
架构需支持 5 倍流量弹性扩展	满足 在修订 3 中已固定

约束条件

已排除：Container Service for Kubernetes (ACK), Container Compute Service (ACS), Distributed Cloud Container Platform for Kubernetes (ACK One)

假设

未记录任何假设。

连接

架构图中各产品之间的交互方式。

从	到	传输内容	类型
外卖 App	云解析 DNS		DNS 解析
外卖 App	ESA	HTTPS	数据
ESA	API 网关	回源	数据
API 网关	微服务集群 (SAE)	REST	数据
微服务集群 (SAE)	MSE Nacos	注册	控制
微服务集群 (SAE)	RocketMQ	事件	异步
微服务集群 (SAE)	Tair 缓存	读写	数据
微服务集群 (SAE)	PolarDB	SQL	数据
微服务集群 (SAE)	OSS	上传	数据
ESA	OSS	图片	数据
微服务集群 (SAE)	日志服务 SLS	日志	控制
微服务集群 (SAE)	ARMS 监控	指标	控制
微服务集群 (SAE)	菜单缓存 (Tair)	读菜单	数据
菜单缓存 (Tair)	PolarDB	回源	数据

修订历史

记录每项请求及其对设计的变更。

修订 1

绘制用时 23.3 秒

请求

我们在做一个外卖应用的后端：大约 12 个微服务、给移动端的 REST API，还有订单事件。团队没有 Kubernetes 经验，请不要使用 Kubernetes。

变更

+ Mobile app · 外卖 App · 在 互联网 内
+ Cloud DNS · 云解析 DNS
+ ESA · 在 边缘加速 内
+ API Gateway · API 网关 · 可用区 A, B
+ WAF · 附加于 API Gateway
+ SAE · 微服务集群 (SAE) · 可用区 A, B
+ MSE · MSE Nacos · 可用区 A, B
+ RocketMQ · 可用区 A, B
+ Tair (Redis) · Tair 缓存 · 可用区 A, B
+ PolarDB · 可用区 A, B
+ OSS · 在 新加坡 (ap-southeast-1) 内
+ SLS · 日志服务 SLS · 在 新加坡 (ap-southeast-1) 内
+ ARMS · ARMS 监控 · 在 新加坡 (ap-southeast-1) 内
+ 12 个连接

修订 2

绘制用时 5.4 秒

请求

为餐厅菜单加一个缓存。

变更

+ Tair (Redis) · 菜单缓存 (Tair) · 可用区 A, B
+ 2 个连接 · 微服务集群 (SAE) to 菜单缓存 (Tair) to PolarDB

修订 3

绘制用时 9.0 秒

请求

预计明年流量会增长 5 倍，请让架构能够扩展。

变更

→ SAE · 微服务集群 (SAE) · 已更改
→ Tair (Redis) · Tair 缓存 · 已更改
→ Tair (Redis) · 菜单缓存 (Tair) · 已更改
→ PolarDB · 已更改
→ RocketMQ · 已更改